Universidad Nacional Mayor de San Marcos · Facultad de Ingeniería Electrónica y Eléctrica

Guion detallado: ¿cómo se almacena un número real en la memoria?

Bits, binario, notación científica e IEEE 754 explicados como un razonamiento: qué decir, qué escribir en la pizarra, demostración y preguntas

Tema: Cómo guarda la computadora un número con decimales (float y double, estándar IEEE 754)

Ejemplos: −6.75 (exacto: C0D80000) y 0.1 (redondeado: 3DCCCCCD)

Duración: Versiones de 20, 12 y 7 minutos

Público: Compañeros que no conocen el tema

- El gancho: 0.1 + 0.2
- Bits, bytes y memoria
- Decimal a binario
- Por qué 0.1 es infinito
- Notación científica binaria
- IEEE 754: signo, exponente y mantisa
- Sesgo y bit implícito
- Ejemplos −6.75 y 0.1
- El redondeo
- Valores especiales y little-endian
- Consecuencias prácticas
- Demostración y preguntas

Índice

Guion detallado: ¿cómo se almacena un número real en la memoria? 1

0. Cómo leer este guion 2
1. La idea central 2
2. Estructura y tiempos 2
3. Antes de exponer 3
   - 3.1 Material 3
   - 3.2 Lista de control 3
4. El guion, bloque por bloque 3
   - Bloque 1 — El gancho: 0.1 + 0.2 3
   - Bloque 2 — Bits y bytes: por qué hay que redondear 4
   - Bloque 3 — El binario con decimales 5
   - Bloque 4 — Convertir 6.75 a binario (pizarra) 5
   - Bloque 5 — El problema: 0.1 es infinito en binario 6
   - Bloque 6 — La idea: notación científica en binario 7
   - Bloque 7 — El estándar IEEE 754 8
   - Bloque 8 — Ejemplo completo: −6.75 como `float` (pizarra) 9
   - Bloque 9 — Ejemplo con trampa: 0.1 como `float` 11
   - Bloque 10 — Volvemos al gancho: por qué 0.1 + 0.2 ≠ 0.3 12
   - Bloque 11 — Valores especiales y orden de los bytes 12
   - Bloque 12 — Consecuencias prácticas 14
   - Bloque 13 — Demostración 15
   - Bloque 14 — Cierre 16
5. Material de apoyo para la pizarra 16
6. Preguntas difíciles y cómo responderlas 17
7. Errores que conviene evitar al explicar 18
8. Tarjeta de bolsillo 18

Guion detallado: ¿cómo se almacena un número real en la memoria?

Este guion sirve para exponer cómo guarda la computadora un número con decimales (un **número real**), pensado para un público que no sabe nada del tema. No se queda en «cómo es» el formato: explica **por qué** es así, paso a paso, para que la exposición se entienda como un razonamiento y no como una lista de datos para memorizar.

Todos los valores que aparecen aquí se comprobaron en Java (las salidas de los programas son reales). La teoría completa, con más detalle, está en el capítulo Parte 1 — Número real en memoria.

> **Consejo**
>
> Si lo que se necesita es **la teoría explicada a fondo** (signo, exponente y sesgo, mantisa, redondeo…), sin indicaciones para exponer, está en ¿Cómo se almacena un número real en la memoria? Teoría completa.

0. Cómo leer este guion

Marca	Significa
Recuadro con «comillas»	Lo que se **dice** en voz alta. No hay que memorizarlo: basta con la idea.
Mostrar:	La figura o la salida que se pone en pantalla.
Pizarra:	Lo que se escribe a mano, paso a paso.
Razonamiento:	El **porqué**. Es lo que convierte la exposición en una explicación y lo que el profesor suele preguntar.
Si preguntan:	Una pregunta probable y su respuesta corta.

1. La idea central

> **En una frase:** la computadora guarda un número real en **notación científica binaria** —**signo, exponente y mantisa**, según el estándar **IEEE 754**— usando una **cantidad fija de bits**; por eso solo puede guardar exactos algunos números, y los demás, como 0.1, se guardan **redondeados**.

Al terminar, el público debería poder:

1. Decir cuántos bytes ocupan un `float` y un `double`, y qué guarda cada parte.
2. Pasar un número como 6.75 a binario.
3. Explicar por qué 0.1 no se puede guardar exacto.
4. Explicar para qué sirven el **sesgo** y el **bit implícito**.
5. Explicar por qué `0.1 + 0.2` da `0.30000000000000004`, y qué hacer al respecto.

2. Estructura y tiempos

Hay tres versiones según el tiempo disponible (los números son minutos):

N.°	Bloque	Completa (20)	Normal (12)	Exprés (7)
1	El gancho: 0.1 + 0.2	1	1	1
2	Bits y bytes: por qué hay que redondear	1.5	1	1
3	El binario con decimales	1	—	—
4	Convertir 6.75 a binario	2	1.5	—
5	El problema: 0.1 es infinito en binario	1.5	1	1.5
6	La idea: notación científica en binario	1.5	1	—
7	El estándar IEEE 754	2	1.5	1.5
8	Ejemplo completo: −6.75	2	1.5	—
9	Ejemplo con trampa: 0.1	1.5	1	—
10	Volvemos al gancho: por qué 0.1 + 0.2 ≠ 0.3	1	1	1
11	Valores especiales y orden de los bytes	1	—	—

N.°	Bloque	Completa (20)	Normal (12)	Exprés (7)
12	Consecuencias prácticas	1	—	—
13	Demostración	2	1	—
14	Cierre	1	0.5	1

- En la versión normal, las consecuencias prácticas (bloque 12) se resumen en el cierre con una frase: «los reales no se comparan con `==`».
- **Si son tres:** A presenta los bloques 1 a 5 (bits y binario); B, del 6 al 10 (IEEE 754 y los ejemplos); C, del 11 al 14 (consecuencias, demostración y cierre).

3. Antes de exponer

3.1 Material

- **Diapositivas:** las figuras de este guion están en la carpeta `diagramas/` (archivos `.svg`, se pueden insertar en PowerPoint o Google Slides).
- **Pizarra:** las cuentas de los bloques 4, 5, 8 y 9 se hacen **a mano**. Conviene escribir antes, en una esquina, la tabla de posiciones del bloque 3 y la tabla binario → hexadecimal de la sección 5.
- **Demostración** (bloque 13), una de dos opciones:
  - **A.** El programa `Numero_Real_en_Memoria` de esta carpeta (NetBeans): hace la conversión paso a paso y la compara con lo que guarda Java.
  - **B.** El programa corto `Main` del bloque 13: se pega en un compilador en línea (por ejemplo OnlineGDB o Programiz) y se ejecuta.

3.2 Lista de control

- ☐ Hacer las conversiones de −6.75 y de 0.1 **a mano, una vez completas**, antes del día de la exposición.
- ☐ Probar la demostración **en la computadora donde se va a exponer**.
- ☐ Agrandar la letra del editor y de la consola.
- ☐ Tener un **plan B**: las salidas reales del bloque 13 en este documento o en capturas.
- ☐ Ensayar con cronómetro y repasar las preguntas de la sección 6.

4. El guion, bloque por bloque

Bloque 1 — El gancho: 0.1 + 0.2

Objetivo: despertar la curiosidad con un resultado que parece imposible.

> «Una pregunta para todos: ¿cuánto es 0.1 más 0.2? … 0.3, claro. Pero miren lo que responde Java:»

Mostrar:

```
0.1 + 0.2 = 0.30000000000000004
0.1 + 0.2 == 0.3 ?  false
```

> «No es un error de Java: pasa igual en C++, en Python, en JavaScript y hasta en Excel. Hoy vamos a ver **por qué**, es decir, cómo se guarda un número con decimales en la memoria. Al final, este resultado va a tener todo el sentido.»

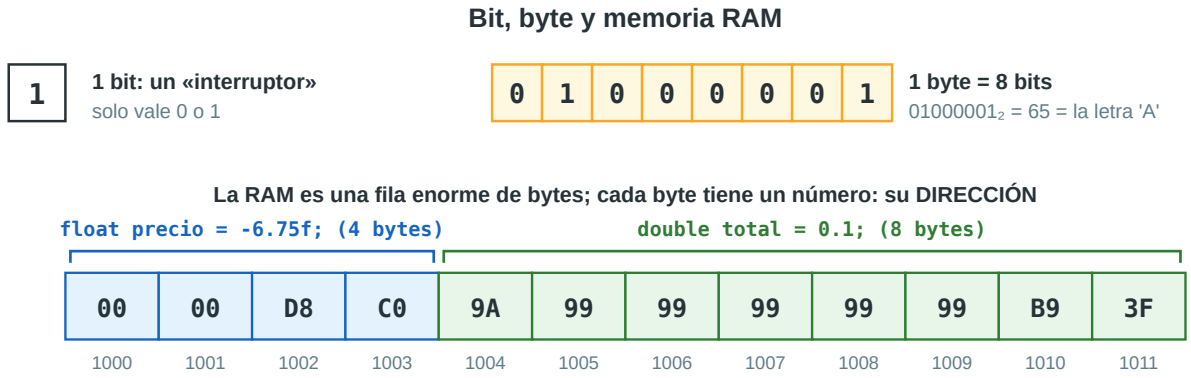
Razonamiento:

- Empezar con una sorpresa crea una **pregunta** en el público, y toda la exposición se convierte en la respuesta.
- No es una falla de un lenguaje: los números reales los maneja el **procesador** siguiendo un estándar internacional, el **IEEE 754**. Por eso todos los lenguajes dan el mismo resultado.

Transición: «Para entenderlo, empecemos por lo más básico: ¿qué hay dentro de la memoria?»

Bloque 2 — Bits y bytes: por qué hay que redondear

Mostrar:



Bit, byte y RAM: un float ocupa 4 bytes y un double 8 bytes

> «La memoria solo guarda **bits**: ceros y unos, como interruptores apagados o encendidos. Ocho bits forman un **byte**. La memoria RAM es como una fila enorme de casilleros de un byte, cada uno con su número de dirección. Un `float` ocupa 4 casilleros, o sea 32 bits; un `double`, 8 casilleros: 64 bits.»

> «Ahora la pregunta clave: con 32 bits, ¿cuántos números distintos se pueden escribir? Dos elevado a 32: unos **4 300 millones** de combinaciones. Parece mucho, pero solo entre 0 y 1 ya hay **infinitos** números reales. Conclusión: es **imposible** guardarlos todos exactos. La computadora elige un conjunto de números que sí puede guardar exactos, y cualquier otro número lo **redondea** al más cercano de ese conjunto.»

Razonamiento:

- Esta es la raíz de todo: **cantidad fija de bits → cantidad finita de valores → el redondeo es inevitable**. No es un defecto de diseño: cualquier forma de escribir números con una cantidad fija de cifras tiene el mismo problema (por ejemplo, escribir 1/3 con cuatro decimales: 0.3333).
- 2^{32} = 4 294 967 296 combinaciones con 32 bits; $2^{64} \approx 1.8 \times 10^{19}$ con 64 bits.
- La verdadera pregunta, entonces, es **qué números elegir** para guardar exactos. El estándar IEEE 754 es la respuesta, y la vamos a construir paso a paso.

Si preguntan: «¿Y los enteros?» → Un `int` usa 32 bits y guarda **exactos** todos los enteros entre −2 147 483 648 y 2 147 483 647, escritos directamente en binario (19 es `00000000 00000000 00000000 00010011`; los negativos usan el método llamado **complemento a dos**). Son exactos, pero no tienen decimales y su rango es limitado.

Bloque 3 — El binario con decimales

> «En decimal, cada posición vale 10 veces la de su derecha: 6.75 es 6 unidades, 7 décimos y 5 centésimos. En binario es igual, pero con el 2: a la izquierda del punto las posiciones valen 1, 2, 4, 8…; a la derecha, **mitades**: 0.5, 0.25, 0.125…»

Pizarra:

Posición	2^3	2^2	2^1	2^0	.	2^{-1}	2^{-2}	2^{-3}	2^{-4}
Vale	8	4	2	1	.	0.5	0.25	0.125	0.0625
110.11_2		1	1	0	.	1	1		

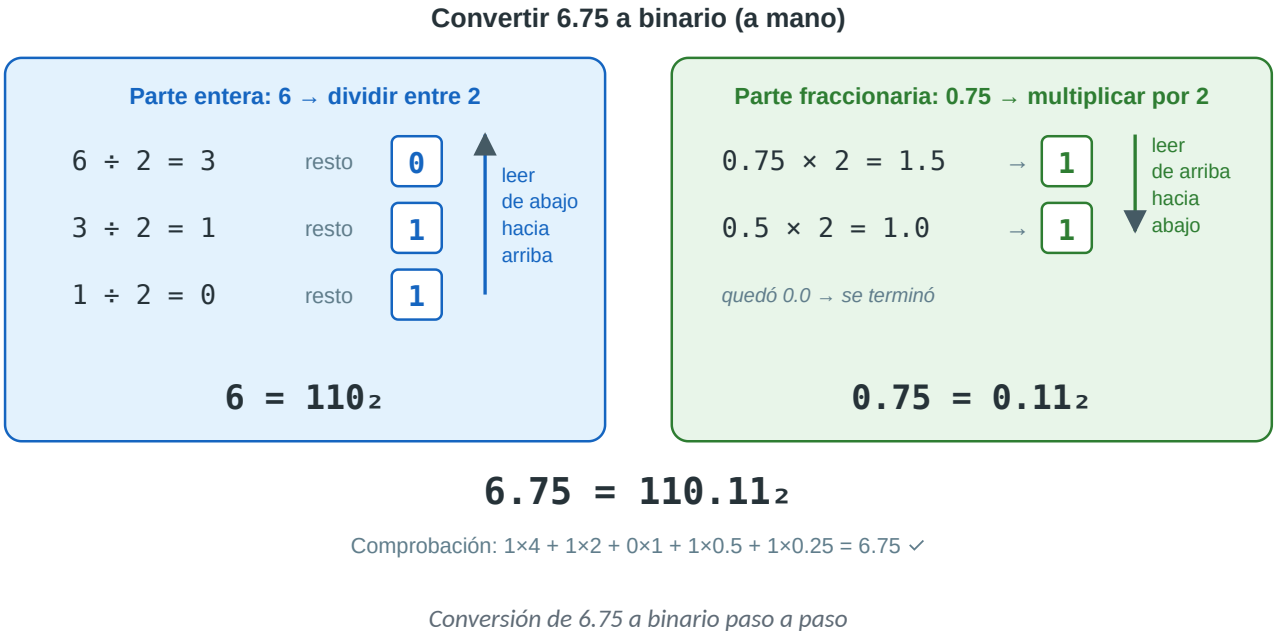
> $110.11_2 = 4 + 2 + 0.5 + 0.25 =$ **6.75**

Razonamiento:

- Después del punto, el binario solo tiene mitades, cuartos, octavos, dieciseisavos… Por lo tanto, **un número es exacto en binario solo si se puede formar sumando esas fracciones**, es decir, si es una fracción cuyo denominador es una potencia de 2.
- 6.75 = 6 + 3/4 → exacto. Esta idea, por sí sola, ya permite **predecir** qué números tendrán error.

Bloque 4 — Convertir 6.75 a binario (pizarra)

Mostrar:



Conversión de 6.75 a binario paso a paso

Pizarra:

```
PARTE ENTERA (6): dividir entre 2      PARTE FRACCIONARIA (0.75): multiplicar por 2
  6 / 2 = 3   resto 0   ↑                0.75 × 2 = 1.5   → 1   │
  3 / 2 = 1   resto 1   │                0.5  × 2 = 1.0   → 1   ↓
  1 / 2 = 0   resto 1   │                (queda 0: termina)
  leer de abajo hacia arriba: 110        leer de arriba hacia abajo: .11

  6.75 = 110.11₂        comprobación: 4 + 2 + 0.5 + 0.25 = 6.75 ✔
```

> «La parte entera se divide entre 2 y se anotan los restos; se leen de abajo hacia arriba: 110. La parte decimal se multiplica por 2 y se anota la parte entera de cada resultado; se leen de arriba hacia abajo: .11. Juntos: 110.11.»

Razonamiento — por qué funcionan los dos métodos:

- **Dividir entre 2:** el resto dice si el número es par (0) o impar (1), y eso es justamente **el último bit**. Al dividir entre 2, ese bit «se cae» y el siguiente pasa a ser el último. Por eso el **primer** resto es el bit **menos** importante y se lee de abajo hacia arriba.
- **Multiplicar por 2:** multiplicar por 2 un número binario **corre el punto un lugar a la derecha** (como multiplicar por 10 en decimal). El dígito que cruza el punto (0 o 1) es el **primer** bit después del punto. Se quita y se repite; por eso se lee de arriba hacia abajo.

Bloque 5 — El problema: 0.1 es infinito en binario

Pizarra:

```
0.1 × 2 = 0.2   → 0
0.2 × 2 = 0.4   → 0
0.4 × 2 = 0.8   → 0
0.8 × 2 = 1.6   → 1     (se quita el 1: queda 0.6)
0.6 × 2 = 1.2   → 1     (queda 0.2)
0.2 × 2 = 0.4   → 0     ← ¡0.2 otra vez! De aquí en adelante todo se repite
0.4 × 2 = 0.8   → 0
0.8 × 2 = 1.6   → 1
...
0.1 = 0.0001100110011001100110011…₂   (el grupo 0011 se repite para siempre)
```

> «Con 0.1 la cuenta **nunca termina**: aparece otra vez 0.2 y desde ahí todo se repite. En binario, 0.1 tiene **infinitos dígitos**, como 1/3 en decimal, que es 0.3333… Como la memoria es finita, hay que **cortar y redondear**. Ese pequeño error es el origen del 0.30000000000000004.»

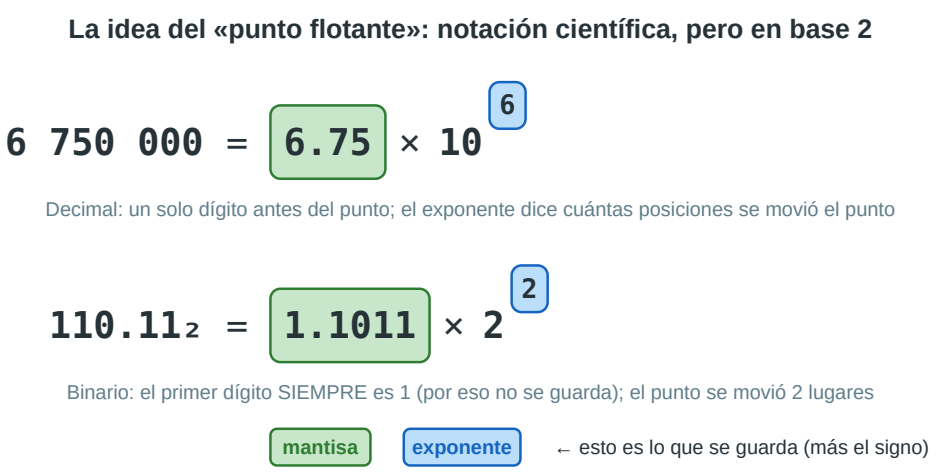
Razonamiento:

- **¿Por qué se repite?** En cada paso la parte decimal solo puede ser 0.0, 0.1, 0.2, …, 0.9 (siempre décimos). Son pocas posibilidades, así que tarde o temprano una se repite, y desde ahí el ciclo vuelve a empezar.
- **¿Qué números son infinitos en binario?** 0.1 = 1/10 = 1/(2 × 5). El binario solo «sabe partir» en mitades; el factor **5** del denominador no se puede formar con mitades, así que el resultado nunca termina. Es la misma razón por la que 1/3 es infinito en decimal: 3 no divide a 10.
- **Consecuencia** (útil para predecir):

Número	¿Exacto en binario?	Por qué
6.75, 0.5, 0.25, 0.125	Sí	Se forman con mitades: 6.75 = 6 + 1/2 + 1/4
4700 y los enteros no muy grandes	Sí	Todo entero se escribe en binario
0.1, 0.2, 0.3, 4.7	**No**	Su denominador tiene un 5: son infinitos en binario

Bloque 6 — La idea: notación científica en binario

Mostrar:



Notación científica decimal y binaria

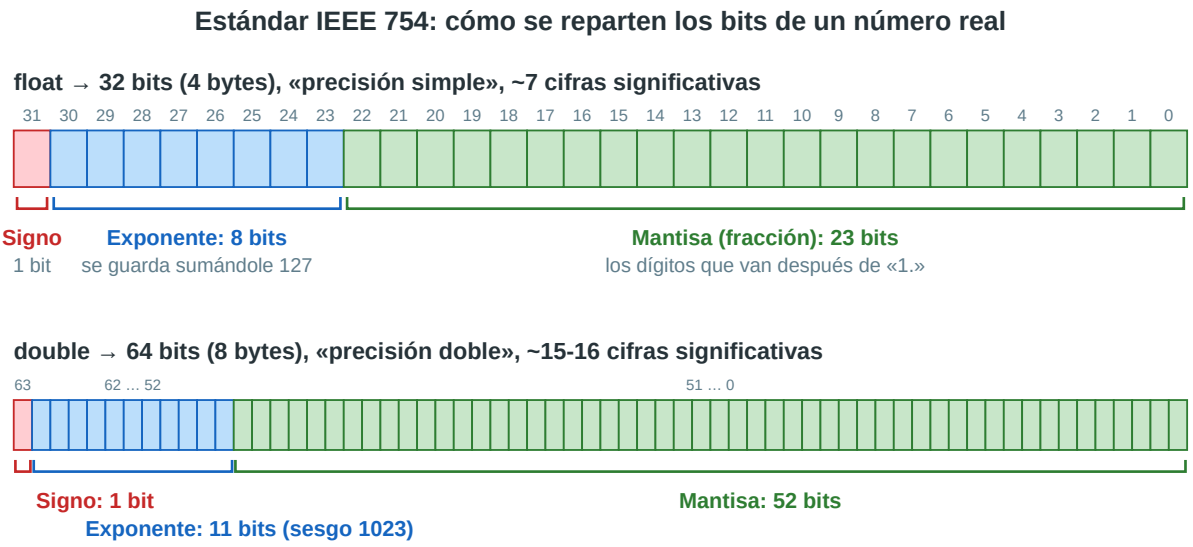
> «Ya sabemos escribir un número en binario. Ahora, ¿cómo lo guardamos en 32 bits? En ciencias, un número como 6 750 000 se escribe 6.75×10^6: unas pocas cifras, la **mantisa**, multiplicadas por una potencia de 10, el **exponente**. La computadora hace lo mismo, pero en base 2: 110.11 se escribe **1.1011×2^2**. Como el punto se mueve según el exponente, se dice que «flota»: de ahí viene el nombre **punto flotante** y el tipo `float`.»

Razonamiento:

- **¿Por qué no guardar el número «tal cual», con el punto en un lugar fijo?** Si se usaran, por ejemplo, 16 bits para la parte entera y 16 para la parte decimal, el número más grande sería 65 535.99… y el paso más pequeño 1/65 536 ≈ 0.000015. No se podría guardar la distancia de la Tierra al Sol (≈ 1.5×10^{11} m) ni la carga de un electrón (≈ 1.6×10^{-19} C).
- La notación científica **separa «qué tan grande es»** (el exponente) **de «cuáles son sus cifras»** (la mantisa). Así, con los mismos 32 bits, un `float` llega desde ≈ 1.2×10^{-38} hasta ≈ 3.4×10^{38}.
- El precio: la cantidad de **cifras significativas es fija** (unas 7 en un `float`). Un número muy grande no puede tener además muchos decimales; por eso, cuanto más grande es el número, más separados están los valores que se pueden guardar (bloque 12).
- **Normalizar** es mover el punto hasta dejar **una sola cifra distinta de cero** antes del punto. En binario la única cifra distinta de cero es el 1, así que **siempre empieza con «1.»**. Si siempre es 1, **no hace falta guardarlo**: es el **bit implícito**, y regala un bit de precisión.

Bloque 7 — El estándar IEEE 754

Mostrar:



$$\text{valor} = (-1)^{\text{signo}} \times 1.\text{mantisa}_2 \times 2^{(\text{exponente} - \text{sesgo})}$$

sesgo = 127 en float · sesgo = 1023 en double · el «1.» no se guarda (bit implícito): ¡se gana 1 bit gratis!

Reparto de bits en float (32) y double (64)

> «Desde 1985, prácticamente todas las computadoras siguen el estándar **IEEE 754**. Un `float` reparte sus 32 bits en tres campos: **1 bit de signo**, **8 de exponente** y **23 de mantisa**. Un `double` hace lo mismo con 64: 1, 11 y 52.»

> «El valor se reconstruye con esta fórmula:»

> **valor = (−1)^signo × 1.mantisa₂ × 2^(exponente guardado − sesgo)**

	`float`	`double`
Tamaño	32 bits = 4 bytes	64 bits = 8 bytes
Signo	1 bit (0 = +, 1 = −)	1 bit
Exponente	8 bits, **sesgo 127**	11 bits, **sesgo 1023**
Mantisa	23 bits (+ 1 implícito = 24)	52 bits (+ 1 implícito = 53)
Cifras confiables	≈ 7	≈ 15–16
Rango aproximado	1.2×10^{-38} a 3.4×10^{38}	2.2×10^{-308} a 1.8×10^{308}

> «El exponente puede ser negativo —0.1 es 1.6×2^{-4}—, pero el campo se guarda **sin signo**. El truco es **sumarle 127** antes de guardarlo: un exponente −4 se guarda como 123, y al leerlo se le resta 127. A ese 127 se le llama **sesgo**.»

Razonamiento:

- **El signo es un solo bit** y solo cambia ese bit: 6.75 es `40D80000` y −6.75 es `C0D80000` (comprobado en Java).
- **¿Por qué sumar un sesgo y no guardar el exponente con signo?** Por dos razones: el campo guarda siempre un número positivo, fácil de manejar; y, como el exponente está en los bits más importantes, **un número más grande tiene un patrón de bits más grande**, de modo que la computadora puede comparar dos reales casi como si fueran enteros. Además, el número 0 queda con **todos sus bits en cero**.
- **¿Por qué 127?** Es la mitad del rango de 8 bits ($2^7 - 1$): reparte los exponentes casi por igual entre negativos y positivos. Los valores 0 y 255 del campo se reservan para casos especiales (bloque 11), así que los exponentes reales van de −126 a +127. En el `double`, $2^{10} - 1 = 1023$.
- **¿Por qué 8 y 23 bits?** Es un **equilibrio**: más bits de exponente dan más **rango** (números más grandes y más pequeños); más bits de mantisa dan más **precisión** (más cifras).
- **¿De dónde salen las «7 cifras»?** La mantisa tiene 24 bits contando el implícito, y $2^{24} = 16\,777\,216 \approx 1.7 \times 10^7$: unas 7 cifras decimales. En el `double`, $2^{53} \approx 9 \times 10^{15}$: unas 16.

Bloque 8 — Ejemplo completo: −6.75 como `float` (pizarra)

Pizarra:

```
1) Signo:        es negativo                          → 1
2) Binario:      6.75 = 110.11₂                        (bloque 4)
3) Normalizar:   110.11 = 1.1011 × 2²                  (el punto se corrió 2 lugares)
4) Exponente:    2 + 127 = 129 = 128 + 1               → 10000001
5) Mantisa:      lo que va después del «1.»: 1011, completado con ceros hasta 23 bits
                                                       → 10110000000000000000000
6) Juntar:       1 | 10000001 | 10110000000000000000000
7) Hexadecimal:  1100 0000 1101 1000 0000 0000 0000 0000
                   C    0    D    8    0    0    0    0   → C0D80000
```

Mostrar:



Signo	1	→ el número es NEGATIVO
Exponente	10000001_2 = 128 + 1 = 129	→ exponente real = 129 − 127 = 2
Mantisa	1011000…0	→ 1.1011_2 = 1 + 1/2 + 0/4 + 1/8 + 1/16 = 1.6875

valor = $(-1)^1 \times 1.6875 \times 2^2 = -1.6875 \times 4 = -6.75$

¡Exacto! 6.75 tiene un binario finito (110.11), así que se guarda sin error.

Los 32 bits de −6.75 y cómo se leen

> «Armemos −6.75. Es negativo: el signo es 1. En binario es 110.11; normalizado, 1.1011 por 2 al cuadrado. El exponente 2 más el sesgo 127 da 129, que en binario es 10000001. La mantisa es lo que va después del "1.": 1011 y ceros hasta completar 23 bits. Juntando todo y agrupando de a cuatro bits, en hexadecimal queda **C0D80000**.»

> «Y lo comprobamos al revés: signo 1, negativo; exponente 129 menos 127, 2; mantisa 1.1011, que es 1.6875. Entonces −1.6875 × 4 = **−6.75**. Exacto, sin ningún error.»

Razonamiento:

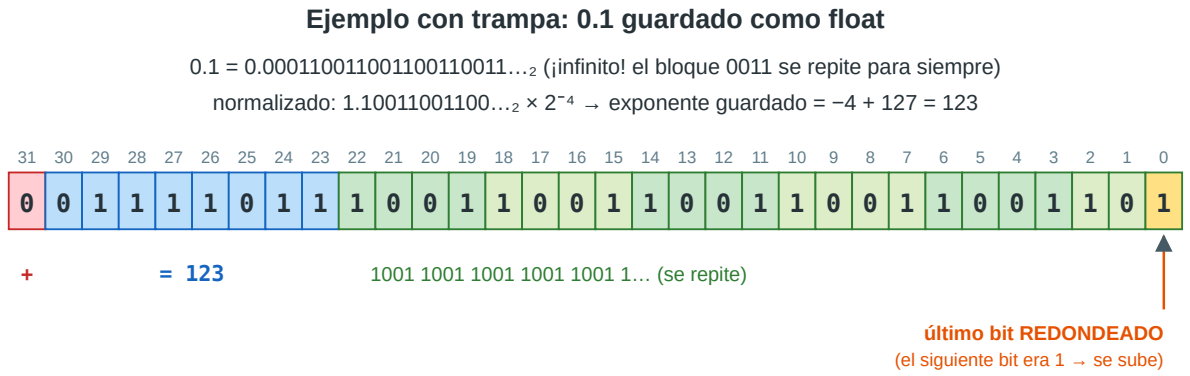
- −6.75 se guarda **exacto** porque 6.75 se forma con mitades (6 + 1/2 + 1/4): su binario termina.
- **¿Para qué el hexadecimal?** 32 ceros y unos son difíciles de leer y de dictar. Cada grupo de 4 bits es exactamente un dígito hexadecimal (0–9 y A–F), así que 32 bits se escriben con 8 caracteres. Es solo una forma corta de escribir los mismos bits.
- **Pasar 129 a binario rápido:** 129 = 128 + 1, y $128 = 2^7$ → `10000001`.

Bloque 9 — Ejemplo con trampa: 0.1 como `float`

Pizarra:

```
1) Signo:        positivo                                    → 0
2) Binario:      0.0001100110011001100110011…₂              (infinito, bloque 5)
3) Normalizar:   1.10011001100110011001100… × 2⁻⁴           (el punto se corrió 4 lugares)
4) Exponente:    −4 + 127 = 123 = 64+32+16+8+2+1             → 01111011
5) Mantisa:      23 bits después del «1.»:   10011001100110011001100 | 1100…
                                                               lo que se corta ↑
                 lo cortado (1100…) es MÁS de la mitad → se redondea hacia arriba
                                                            → 10011001100110011001101
6) Juntar:       0 | 01111011 | 10011001100110011001101
7) Hexadecimal:  0011 1101 1100 1100 1100 1100 1100 1101   → 3DCCCCCD
```

Mostrar:



Lo que REALMENTE queda guardado:

`float : 0.100000001490116119384765625`

`double: 0.1000000000000000055511151231257827…`

Ninguno es exactamente 0.1. Java imprime «0.1» porque muestra la forma más corta que lo identifica.

Por eso en la computadora 0.1 + 0.2 da 0.30000000000000004

Los 32 bits de 0.1: el patrón se repite y el último bit se redondea

> «Con 0.1 la mantisa no termina, así que hay que cortarla en 23 bits. Lo que se corta empieza con 1100…, que es más de la mitad del último bit, así que se **redondea hacia arriba**: el último 0 pasa a 1. Por eso lo que realmente queda guardado no es 0.1, sino:»

> **0.100000001490116119384765625** (en `float`)

> «Un poquito **más** que 0.1, porque se redondeó hacia arriba. En un `double` el error es muchísimo más pequeño, pero también existe: se guarda 0.1000000000000000055511151231257827…»

Razonamiento:

- **La regla de redondeo** es «al más cercano»: se mira lo que se corta. Si es más de la mitad del último bit, se sube; si es menos, se deja; si es **exactamente** la mitad, se elige el valor cuyo último bit sea 0 (se llama «redondeo al par»).

- Por esa regla, el error **nunca supera media unidad del último bit**: en un `float`, como mucho una parte en 16 millones ($2^{-24} \approx 6 \times 10^{-8}$ del valor); aquí, el error es 0.000000001490116… En un `double`, como mucho $2^{-53} \approx 1.1 \times 10^{-16}$ del valor.
- **¿Por qué Java muestra «0.1» si guarda otra cosa?** Java imprime el número decimal **más corto** que, al volver a leerlo, da exactamente el mismo valor guardado. «0.1» es una etiqueta abreviada de ese valor, pero **por dentro no es 0.1**.

Bloque 10 — Volvemos al gancho: por qué 0.1 + 0.2 ≠ 0.3

> «Ahora sí podemos explicar el misterio del inicio. Usemos `double`, que es lo que usa Java por defecto:»

	Lo que realmente queda guardado (`double`)
0.1	0.1000000000000000055511151231257827… (un poquito **más**)
0.2	0.2000000000000000111022302462515654… (un poquito **más**)
Suma exacta de esos dos valores	0.3000000000000000166533453693773481…
Resultado guardado (redondeado)	0.3000000000000000444089209850062616… → Java lo muestra como **0.30000000000000004**
El número 0.3 escrito directamente	0.2999999999999999888977697537484345… (un poquito **menos**)

> «0.1 y 0.2 ya estaban guardados un poquito pasados. Al sumarlos, el resultado tampoco se puede guardar exacto, así que se redondea al `double` más cercano, y ese `double` **no es el mismo** en el que queda guardado 0.3. Por eso, para la computadora, `0.1 + 0.2 == 0.3` es **falso**.»

Razonamiento (para quien quiera el detalle fino):

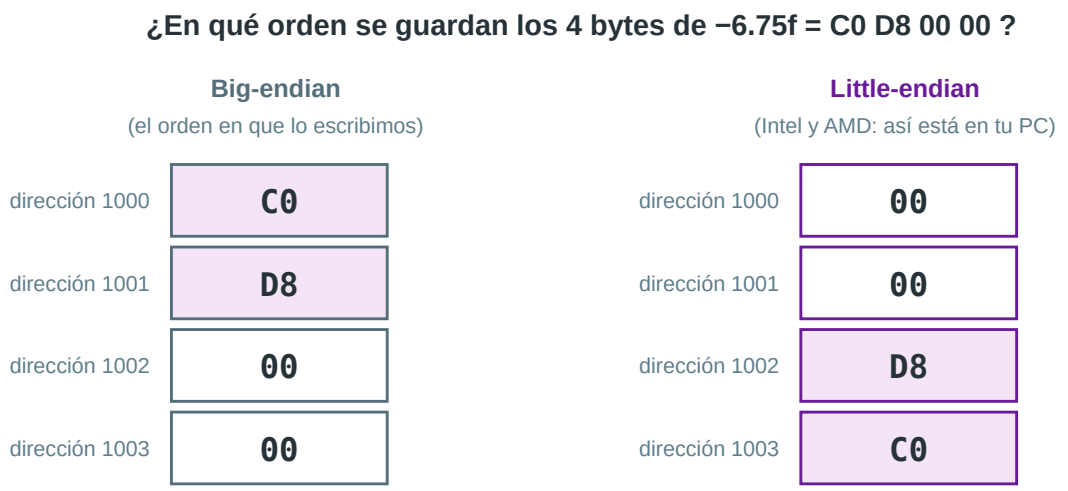
- La suma exacta, 0.30000000000000001665…, cae **justo a la mitad** entre dos `double` vecinos: 0.29999999999999998889… (el de 0.3) y 0.30000000000000004440…. Es un empate, y la regla de «redondeo al par» elige el de arriba. Resultado: queda **un paso** por encima del 0.3 guardado; la diferencia es 5.55×10^{-17}.
- **Lección práctica:** los reales no se comparan con `==`, sino con una **tolerancia**: `Math.abs(a - b) < 1e-9` (¿están más cerca que una milmillonésima?).
- A veces los errores **se compensan**: `0.1 + 0.4 == 0.5` da `true`, y en `float`, `0.1f + 0.2f == 0.3f` también da `true`. Como no se puede saber de antemano cuándo pasa, **nunca** hay que confiar en `==` con reales.

Bloque 11 — Valores especiales y orden de los bytes

> «Los valores extremos del campo de exponente, todo ceros y todo unos, se reservan para casos especiales:»

Exponente	Mantisa	Significa	En Java
Todo ceros	Cero	**Cero** (hay +0 y −0)	`0.0` y `-0.0`
Todo ceros	Distinta de cero	**Subnormales**: números aún más pequeños que el mínimo normal	`Float.MIN_VALUE` = 1.4×10^{-45}
Todo unos	Cero	**Infinito** (+∞ o −∞)	`1.0 / 0` → `Infinity`
Todo unos	Distinta de cero	**NaN** (*Not a Number*, «no es un número»)	`0.0 / 0` → `NaN`

Mostrar:



Big-endian vs little-endian para −6.75f

> «Un detalle más: los 4 bytes de `C0D80000` se guardan en la RAM **al revés**: primero `00`, luego `00`, `D8` y `C0`. Las PC con procesador Intel o AMD, y la mayoría de los celulares, guardan primero el byte menos importante: se llama **little-endian**. Java lo oculta: siempre vemos el número completo.»

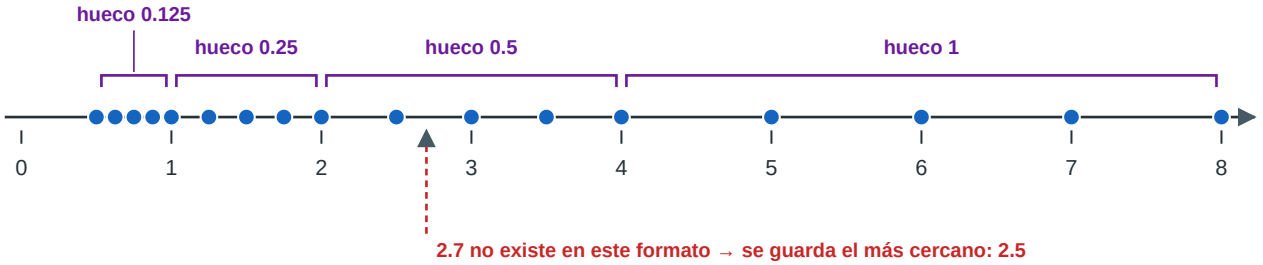
Razonamiento:

- Reservar esos patrones permite que operaciones como `1.0 / 0` den un resultado definido (`Infinity`) en lugar de detener el programa. Curiosidad: `NaN == NaN` es **falso** (un «no número» no es igual a nada), y `-0.0 == 0.0` es verdadero aunque sus bits son distintos.
- Los subnormales «rellenan» el hueco entre el cero y el número normal más pequeño, para que los valores diminutos no salten bruscamente a cero.

Bloque 12 — Consecuencias prácticas

Mostrar:

Los números que SÍ se pueden guardar no están repartidos de forma pareja



Formato «de juguete» con solo 2 bits de mantisa. El float usa 23 y el double 52 (puntos mucho más juntos), pero el patrón es el mismo: cada vez que el número se duplica, el hueco entre valores posibles también se duplica.

Los huecos entre valores representables crecen con el número

> «¿Qué significa todo esto cuando programamos? Cinco cosas:»

1. **La precisión es limitada.** Un `float` tiene unas 7 cifras: **16 777 217** guardado como `float` queda **16 777 216**. Por eso Java y C++ usan `double` por defecto.
2. **Los errores se acumulan.** Sumar 0.1 diez veces da **0.9999999999999999**, no 1.0.
3. **No comparar reales con `==`**: usar una tolerancia (bloque 10).
4. **Dinero:** los bancos y las tiendas no usan `double`; usan céntimos enteros o tipos decimales exactos (`BigDecimal` en Java).
5. **Los huecos crecen:** después de 1.0, el siguiente `float` está a 0.00000012; después de 1 000 000, a 0.0625. Con números grandes se pierden decimales.

Razonamiento: todas son consecuencia de lo mismo: **cantidad fija de cifras significativas** (mantisa) y **redondeo** de lo que no cabe.

> **Nota**
>
> **Conexión con el programa de las bandas:** ese programa solo usa enteros (`int`), que se guardan exactos, así que no tiene este problema. Aparecería si se calculara el **valor** de la resistencia con la banda dorada, que multiplica por 0.1: `12 * 0.1` da **1.2000000000000002**, mientras que `12 / 10.0` da **1.2**. Por eso, en esa ampliación, conviene dividir entre 10 en lugar de multiplicar por 0.1.

Bloque 13 — Demostración

Opción A — El programa `Numero_Real_en_Memoria` (NetBeans):

Escribir	Qué se ve	Qué decir
`1` y luego `-6.75`	La conversión paso a paso y los bits del `float` (`1` , `10000001` , `10110000000000000000000`), hexadecimal `C0D80000`	«Es lo mismo que hicimos en la pizarra, y abajo dice que **coincide** con lo que Java guarda.»
`1` y luego `0.1`	El patrón 0011 que se repite, el redondeo del bit 24 y el «Valor EXACTO guardado: 0.1000000014901161193847656 25»	«Aquí se ve el redondeo y el error.»
`2`	Las curiosidades: 0.1 + 0.2, la suma de 0.1 diez veces, el límite del `float` , infinito y NaN	«Cada una es una de las consecuencias que vimos.»
`0`	Fin del programa	

Las salidas completas están en el capítulo Parte 1 — Número real en memoria (sección 13).

Opción B — Programa corto `Main` (se pega tal cual en un compilador en línea):

```
import java.math.BigDecimal;

public class Main
{
    // Muestra los 32 bits de un float separados en: signo | exponente | mantisa
    static void verBits(float f)
    {
        int bits = Float.floatToIntBits(f);   // los 32 bits tal como estan en la memoria
        String b = String.format("%32s", Integer.toBinaryString(bits)).replace(' ', '0');
        System.out.println("   " + f + " -> " + b.substring(0, 1) + " | " + b.substring(1, 9)
                           + " | " + b.substring(9) + "   hex: "
                           + Integer.toHexString(bits).toUpperCase());
    }

    public static void main(String[] args)
    {
        System.out.println("1) 0.1 + 0.2 = " + (0.1 + 0.2));
        System.out.println("   0.1 + 0.2 == 0.3 ? " + (0.1 + 0.2 == 0.3));

        System.out.println("2) Los bits de un float (signo | exponente | mantisa):");
        verBits(-6.75f);
        verBits(0.1f);

        System.out.println("3) Lo que REALMENTE queda guardado:");
        System.out.println("   0.1 como float : " + new BigDecimal(0.1f));
        System.out.println("   0.1 como double: " + new BigDecimal(0.1));

        double suma = 0;
        for(int i = 1; i <= 10; i++)
            suma = suma + 0.1;
        System.out.println("4) 0.1 sumado 10 veces = " + suma);

        float grande = 16777217;
        System.out.println("5) 16777217 guardado como float = " + (int) grande);

        System.out.println("6) 1.0 / 0 = " + (1.0 / 0) + "    0.0 / 0 = " + (0.0 / 0));

        System.out.println("7) Comparar con tolerancia: |(0.1 + 0.2) - 0.3| < 1e-9 ? "
                           + (Math.abs((0.1 + 0.2) - 0.3) < 1e-9));
    }
}
```

Salida real:

```
1) 0.1 + 0.2 = 0.30000000000000004
   0.1 + 0.2 == 0.3 ? false
2) Los bits de un float (signo | exponente | mantisa):
   -6.75 -> 1 | 10000001 | 10110000000000000000000   hex: C0D80000
   0.1 -> 0 | 01111011 | 10011001100110011001101   hex: 3DCCCCCD
3) Lo que REALMENTE queda guardado:
   0.1 como float : 0.100000001490116119384765625
   0.1 como double: 0.1000000000000000055511151231257827021181583404541015625
4) 0.1 sumado 10 veces = 0.9999999999999999
5) 16777217 guardado como float = 16777216
6) 1.0 / 0 = Infinity    0.0 / 0 = NaN
7) Comparar con tolerancia: |(0.1 + 0.2) - 0.3| < 1e-9 ? true
```

> «Línea 1: el gancho del inicio. Línea 2: los bits de −6.75 y de 0.1, idénticos a los que armamos en la pizarra: C0D80000 y 3DCCCCCD. Línea 3: lo que de verdad queda guardado cuando escribimos 0.1. Las líneas 4 a 6 son las consecuencias: los errores se acumulan, el `float` tiene un límite de precisión, y dividir entre cero da infinito o NaN. Y la línea 7, la forma correcta de comparar reales: con tolerancia.»

Razonamiento — qué hace el programa corto: `Float.floatToIntBits(f)` le pide a Java los **32 bits exactos** del `float`, tal como están en la memoria; luego se separan en signo, exponente y mantisa. `new BigDecimal(0.1f)` muestra el valor guardado **con todos sus decimales**, sin el redondeo que hace Java al imprimir.

Bloque 14 — Cierre

> «Para cerrar, cinco ideas:
>
> 1. La computadora guarda todo en bits: un `float` usa 32 y un `double`, 64. Con una cantidad fija de bits solo se pueden guardar exactos algunos números.
> 2. Los reales se guardan en **notación científica binaria**: signo, exponente y mantisa, según el estándar **IEEE 754**.
> 3. Se normaliza a la forma 1.algo × 2^exponente; el «1.» no se guarda (**bit implícito**) y al exponente se le suma un **sesgo** (127 en `float`).
> 4. Números como 0.1 son **infinitos en binario**: se cortan y se redondean. Por eso 0.1 + 0.2 da 0.30000000000000004.
> 5. En la práctica: usar `double`, no comparar reales con `==` y no usar `double` para el dinero.
>
> Muchas gracias. ¿Preguntas?»

5. Material de apoyo para la pizarra

Tabla binario → hexadecimal (cada 4 bits son un dígito):

Bits	0000	0001	0010	0011	0100	0101	0110	0111
Hex	0	1	2	3	4	5	6	7

Bits	1000	1001	1010	1011	1100	1101	1110	1111
Hex	8	9	A	B	C	D	E	F

Potencias de 2 útiles:

2^{-4}	2^{-3}	2^{-2}	2^{-1}	2^{0}	2^{1}	2^{2}	2^{3}	2^{4}	2^{5}	2^{6}	2^{7}	2^{8}	2^{10}	2^{24}	2^{32}
0.0625	0.125	0.25	0.5	1	2	4	8	16	32	64	128	256	1 024	16 777 216	4 294 967 296

Comprobación de 0.1 (cómo se lee el resultado): 0 | 01111011 | 10011001100110011001101 → signo +, exponente 123 − 127 = −4, mantisa $1.10011001100110011001101_2$ → $1.6000000238... \times 2^{-4}$ = **0.100000001490116...**

Otro ejemplo para practicar: 4700 = 1001001011100_2 = $1.001001011100 \times 2^{12}$ → exponente 12 + 127 = 139 = `10001011` → `0 | 10001011 | 00100101110000000000000` = `4592E000` (exacto, porque es entero).

6. Preguntas difíciles y cómo responderlas

Pregunta	Respuesta
¿Por qué no guarda el número en decimal, como lo escribimos?	Los circuitos trabajan con dos estados (encendido/apagado), así que todo se guarda en binario. Existen formatos decimales (por ejemplo `BigDecimal` en Java), pero son más lentos porque se calculan por programa.
¿Qué números decimales se guardan exactos?	Los que son fracciones con denominador potencia de 2: 0.5, 0.25, 0.75, 6.75... y los enteros no muy grandes. Los que tienen un 5 en el denominador (0.1 = 1/10) no.
¿Por qué el exponente usa sesgo?	Para guardarlo como número sin signo y para que los números más grandes tengan patrones de bits más grandes (se comparan fácilmente). El cero queda con todos los bits en 0.
¿Qué es el bit implícito y no se pierde información?	Al normalizar, el primer dígito siempre es 1; como se sabe que está ahí, no se guarda. No se pierde nada y se gana un bit de precisión: 23 bits guardados valen como 24.
¿Por qué el `float` tiene unas 7 cifras y el `double` unas 16?	Por los bits de mantisa: $2^{24} \approx 1.7 \times 10^{7}$ (7 cifras) y $2^{53} \approx 9 \times 10^{15}$ (15–16 cifras).
¿Por qué Java muestra 0.1 si guarda otra cosa?	Muestra el decimal más corto que identifica al valor guardado. Con `new BigDecimal(0.1)` se ve el valor completo.
¿Por qué 0.1 + 0.2 no da 0.3, pero 0.1 + 0.4 sí da 0.5?	En los dos casos hay redondeo, pero a veces los errores se compensan y el resultado cae justo en el valor esperado. No se puede predecir fácilmente: por eso no se usa `==`.
¿Cómo se comparan dos reales?	Con tolerancia: `Math.abs(a - b) < 1e-9`.
¿Qué tipo se usa para el dinero?	Céntimos en un entero (`long`) o `BigDecimal`, que hace cuentas decimales exactas.
¿Qué pasa al dividir entre cero?	Con reales no hay error: `1.0 / 0` da `Infinity` y `0.0 / 0` da `NaN`. (Con enteros, `1 / 0` sí detiene el programa.)
¿ `NaN` es igual a `NaN` ?	No: `NaN == NaN` es `false`. Para detectarlo se usa `Double.isNaN(x)`.
¿Existe el −0?	Sí: tiene el bit de signo en 1 y todo lo demás en 0. Aun así, `-0.0 == 0.0` es verdadero.
¿En C++ es igual?	Sí: `float` y `double` siguen IEEE 754 en prácticamente todas las computadoras; `0.1 + 0.2` da el mismo resultado.

Pregunta	Respuesta
¿Qué es little-endian?	Guardar primero (en la dirección más baja) el byte menos importante: −6.75f (`C0 D8 00 00`) queda en la RAM como `00 00 D8 C0` .
¿Cuál es el `float` más grande?	≈ 3.4×10^{38} (`Float.MAX_VALUE`); el `double` llega a ≈ 1.8×10^{308}.
¿4.7 se guarda exacto?	No: como `double` queda 4.70000000000000017763568394002504646778106689453125, y como `float` , 4.69999980926513671875.
¿Hasta dónde un `float` guarda enteros exactos?	Hasta 2^{24} = 16 777 216. Después ya no alcanzan los bits: 16 777 217 se guarda como 16 777 216.

7. Errores que conviene evitar al explicar

- Decir que «la computadora se equivoca»: **redondea**, siguiendo una regla exacta y conocida.
- Confundir la **mantisa** con «la parte decimal del número»: son las cifras que van después del «1.» del número **normalizado**.
- Olvidar sumar el **sesgo** al exponente (o restarlo al leer).
- Escribir el «1.» dentro de la mantisa: **no se guarda**.
- Leer los restos de la división **de arriba hacia abajo** (es al revés).
- Decir que el `double` «no tiene error»: tiene mucho menos, pero también redondea.
- Usar en la demostración números que no se probaron antes.

8. Tarjeta de bolsillo

Tema	Lo esencial
Idea	Notación científica binaria con bits fijos → algunos números se redondean
`float`	32 bits: signo 1 · exponente 8 (sesgo 127) · mantisa 23 (+1 implícito) · ≈ 7 cifras
`double`	64 bits: signo 1 · exponente 11 (sesgo 1023) · mantisa 52 (+1 implícito) · ≈ 16 cifras
Fórmula	valor = (−1)^signo × 1.mantisa × 2^(exponente − sesgo)
Pasar a binario	Entera: ÷2, restos de abajo hacia arriba · Decimal: ×2, partes enteras de arriba hacia abajo
−6.75	$110.11_2 = 1.1011 \times 2^2$ → exp. 129 → `1 10000001 1011000…` → `C0D80000` (exacto)
0.1	$0.000110011\ldots_2$ (infinito) → exp. 123 → redondeo → `3DCCCCCD` → 0.100000001490116…
0.1 + 0.2	0.30000000000000004 → comparar con `Math.abs(a - b) < 1e-9`
Especiales	Exponente todo 0 → cero y subnormales · todo 1 → infinito y NaN
RAM	Little-endian: `C0D80000` queda como `00 00 D8 C0`